IThub

Понятие объекта, класса. Объявление класса



ООП: КЛАСС И ОБЪЕКТ

Можно провести следующую аналогию: у нас у всех есть некоторое представление о человеке - наличие двух рук, двух ног, головы, туловища и т.д.

КЛАСС

Шаблон “Человек”



Характеристики (поля)

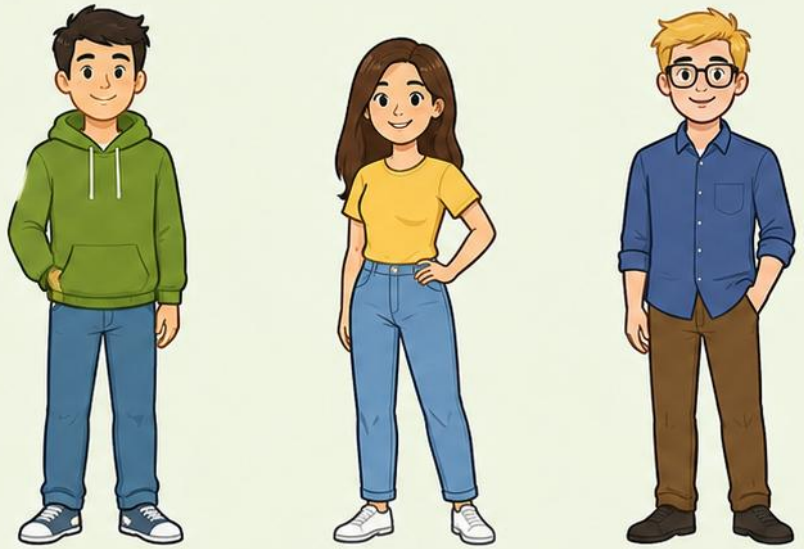
- имя
- возраст
- рост
- цвет глаз и т.д.

Поведение (методы)

- говорить()
- ходить()
- думать()
- работать() и т.д.

ОБЪЕКТЫ (ЭКЗЕМПЛЯРЫ КЛАССА)

Реальные люди



Иван
- возраст: 20
- рост: 180 см
- цвет глаз: карий
- говорит()
- ходит()
- работает() и т.д.

Мария
- возраст: 25
- рост: 165 см
- цвет глаз: зелёный
- говорит()
- ходит()
- работает()и т.д.

Алексей
- возраст: 30
- рост: 175 см
- цвет глаз: голубой
- говорит()
- ходит()
- работает() и т.д.

Класс – это шаблон, описание того, какими свойствами и поведением обладает человек.
Объект – это конкретный человек, созданный на основе этого шаблона класса.

IThub

Основы классов

```
class <имя класса> {
    тип переменная-экземпляра1;
    тип переменная-экземпляра2;
     …
    тип имя-метода1(список параметров) {
    // тело метода
    }

    тип имя-метода2(список параметров) {
    // тело метода
    }
    …
}
```

- Класс объявляется с применением ключевого слова class
- Данные, или переменные, определенные в классе, называются переменными экземпляра (для хранения состояния объекта)
- В совокупности методы и переменные, определенные в классе, называются членами класса

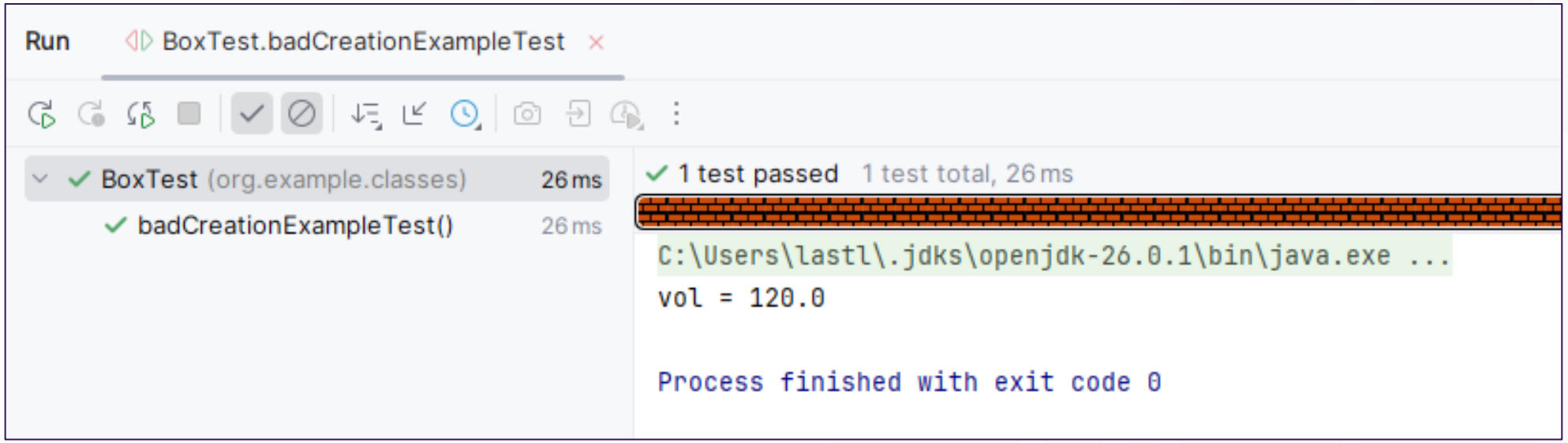
IThub

```java
public class Box {

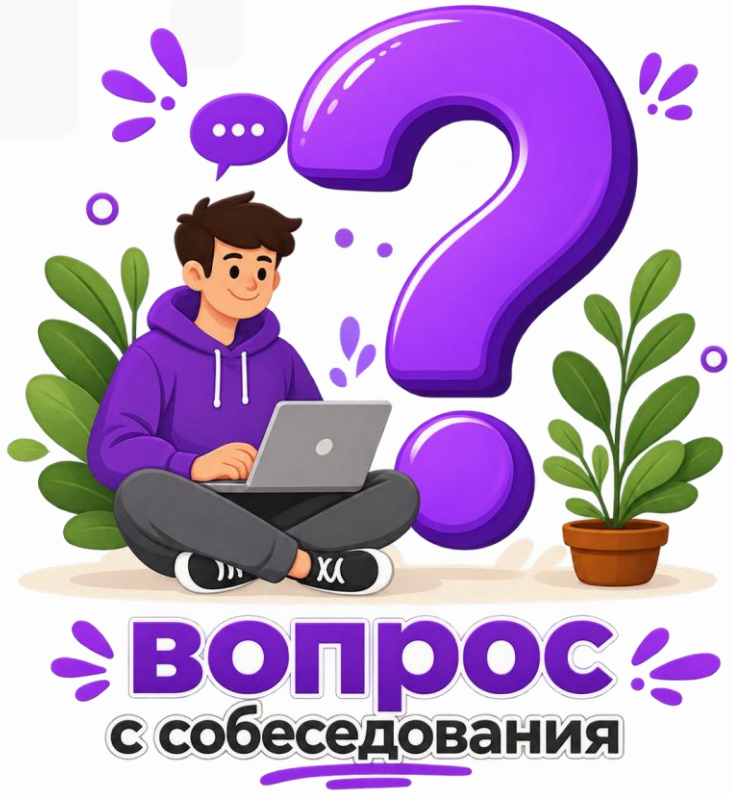
    public double width;
    public double height;
    public double depth;
}
```

```java
public void badCreationExampleTest() {
    Box box = new Box();
    box.height = 15;
    box.width = 2;
    box.depth = 4;
    double vol = box.height * box.width * box.depth;

    System.out.println("vol = " + vol);
}
```



IThub



Что такое конструктор?

Конструктор — это специальный метод, у которого отсутствует возвращаемый тип и который имеет то же имя, что и класс, в котором он используется.
Конструктор вызывается при создании нового объекта класса и определяет действия необходимые для его инициализации.

Конструктор

- инициализирует объект немедленно после создания
- синтаксически похож на метод
- нет возвращаемого типа, даже void - неявным возвращаемым типом конструктора класса является сам класс
- **this** – используется внутри метода для ссылки на текущий объект (можно использовать для устранения любых конфликтов имен)
- конструкторы также можно перегружать

IThub

```java
public class Box {

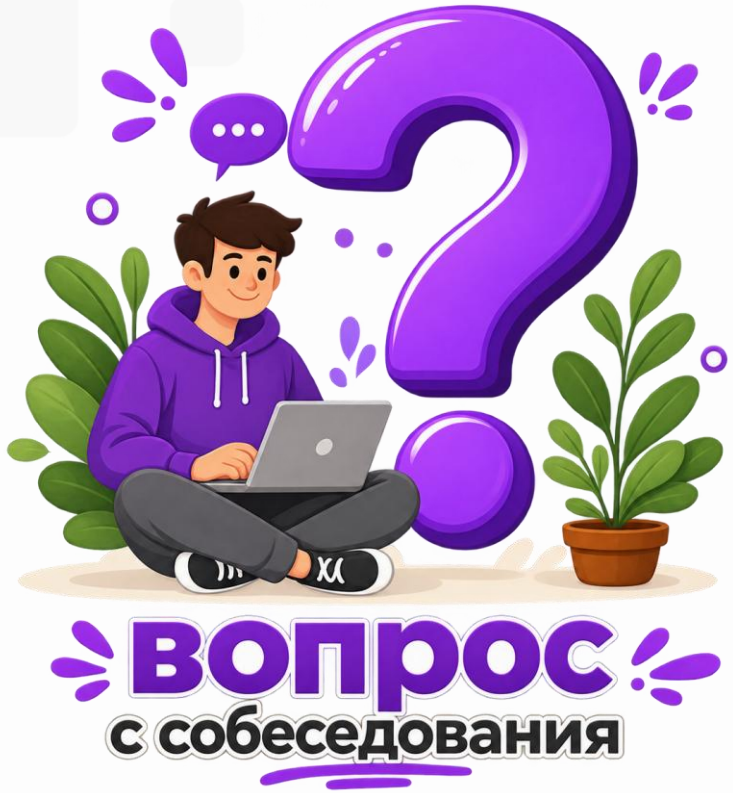
    private double width;
    private double height;
    private double depth;

    public Box() {
        this.width = 0.0;
        this.height = 0.0;
        this.depth = 0.0;
    }

    public Box (double value) {
        this.width = value;
        this.height = value;
        this.depth = value;
    }

    public Box(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }
```

IThub



Что такое «конструктор по умолчанию»?

Если у какого-либо класса не определить конструктор, то компилятор сгенерирует конструктор без аргументов - так называемый «конструктор по умолчанию».
Если у класса уже определен какой-либо конструктор, то конструктор по умолчанию создан не будет и, если он необходим, его нужно описывать явно.

IThub

Блоки инициализации (инициализаторы)

Блоки инициализации представляют собой код, заключенный в фигурные скобки и размещаемый внутри класса **вне объявления методов или конструкторов**.

- Существуют статические и нестатические блоки инициализации.
- Блок инициализации выполняется **перед инициализацией класса** загрузчиком классов или созданием объекта класса с помощью конструктора.
- Несколько блоков инициализации выполняются в порядке следования в коде класса.
- Блок инициализации способен генерировать исключения, если их объявления перечислены в throws всех конструкторов класса.
- Блок инициализации возможно создать и в анонимном классе.

```
public class TestClass {

    public static String v = "Some val";

    {
        System.out.println("!!! Non-static initializer");
    }

    static {
        System.out.println("!!! Static initializer");
    }

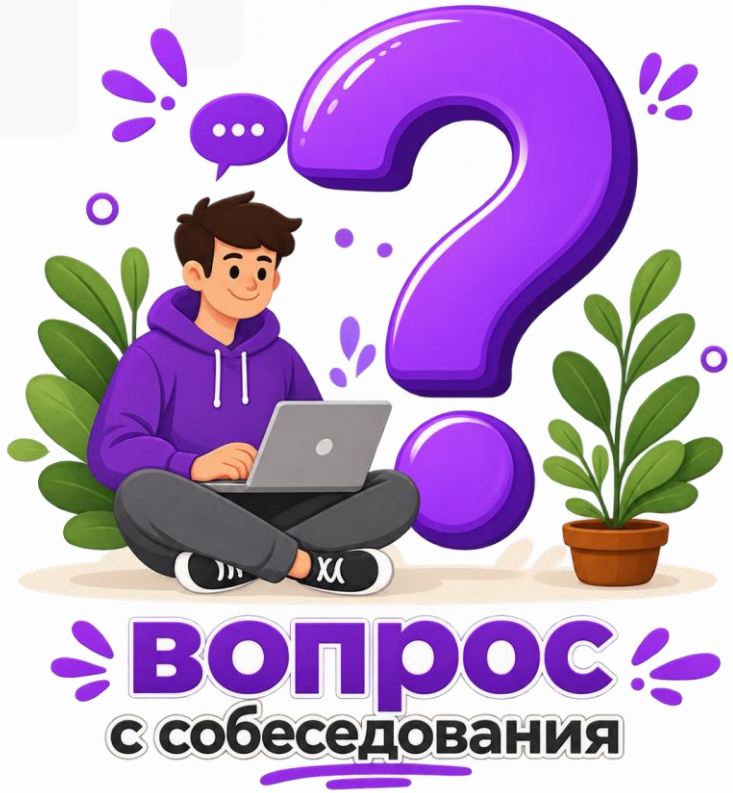
    public void a() {
        System.out.println("!!! a() called");
    }

}
```

```
 public static void main(String[] args) {

        TestClass testClass = new TestClass();
        testClass.a();
}
```

```
!!! Static initializer
!!! Non-static initializer
!!! a() called
```



Каков порядок вызова конструкторов и блоков инициализации с учётом иерархии классов?

Сначала вызываются все статические блоки в очередности от первого статического блока корневого предка и выше по цепочке иерархии до статических блоков самого класса.

Затем вызываются нестатические блоки инициализации корневого предка, конструктор корневого предка и так далее вплоть до нестатических блоков и конструктора самого класса.

Ключевое слово this

Ключевое слово **this** — это ссылка на текущий объект, то есть объект, внутри которого сейчас выполняется метод или конструктор. Используется в конструкторе, где параметры называются так же, как и поля класса. И чтобы разграничить поля и параметры, применяется ключевое слово this.

С помощью this можно в конструкторе вызвать другой конструктор класса и передать для его параметров необходимые значения.

```
public Box(double width, double height, double depth) {
    this.width = width;
    this.height = height;
    this.depth = depth;
}
```

Классы

- классы должны быть компактными
- классы должны быть еще компактнее
- имя класса должно описывать его ответственности
- принцип единой ответственности (SRP) утверждает, что класс должен иметь одну - и только одну - причину для изменения
- классы должны иметь небольшое количество переменных экземпляров



IThub

Геттеры

```
public double getWidth() {
    return width;
}

public double getHeight() {
    return height;
}

public double getDepth() {
    return depth;
}
```

Геттер — это метод, назначение которого - возвращать (от слова *get*) значение поля.

Чтобы никто не смог поменять данные об объекте на какие-то неверные значения мы меняем модификаторы доступа полей с public на private. Данные "в безопасности", но получить значение мы не можем. В таком случае на помощь приходят – геттеры.

IThub

Сеттеры

```
public void setWidth(double width) {
    this.width = width;
}

public void setHeight(double height) {
    this.height = height;
}

public void setDepth(double depth) {
    this.depth = depth;
}
```

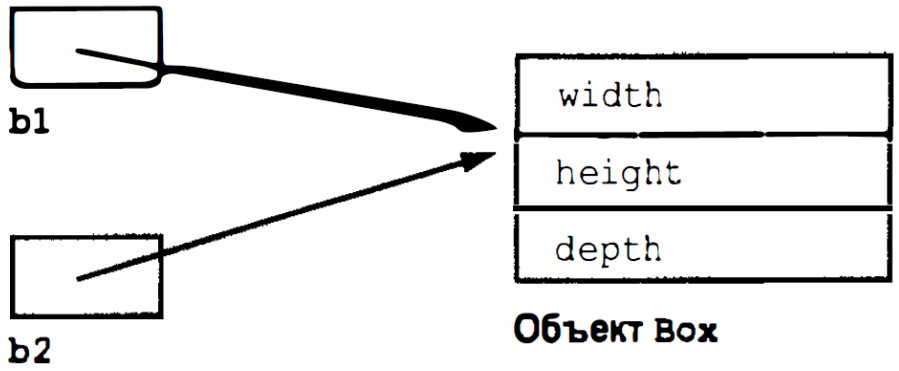
Сеттер — это метод, предназначенный для изменения (установки; от слова set) значения поля

Присваивание для переменных ссылок на объекты

```
@Test
public void linksExampleTest() {
    Box b1 = new Box(15, 2, 4);
    Box b2 = b1;

    Assertions.assertEquals(15, b2.getWidth(), 0.0);

    b1.setWidth(30);
    Assertions.assertNotEquals(15, b2.getWidth(), 0.0);
    Assertions.assertEquals(30, b2.getWidth(), 0.0);
}
```



IThub

IThub

Значение null

- null означает, что переменная не ссылается ни на какой объект
- Объекта нет - переменная существует, но пока не содержит ссылку на объект.
- Ссылка отсутствует - null можно присвоить переменной ссылочного типа.
- Осторожно! Попытка обратиться к объекту через null приведёт к ошибке NullPointerException.
- Может быть возвращен из метода.

1. Инициализация переменной без объекта

Можно создать переменную, но пока не создавать объект:

```
String name = null;
```

Сейчас переменная name существует, но не указывает ни на какой объект.

2. Проверка наличия объекта

Перед использованием объекта часто проверяют, не равен ли он null:

```
if (name != null) {
    System.out.println(name.length());
}
```

Это помогает избежать ошибок.

3. Отсутствие значения в объектах

Например, у пользователя может не быть указанного адреса:

```
class User {
    String name;
    String address;
}
User user = new User();
user.address = null;
```

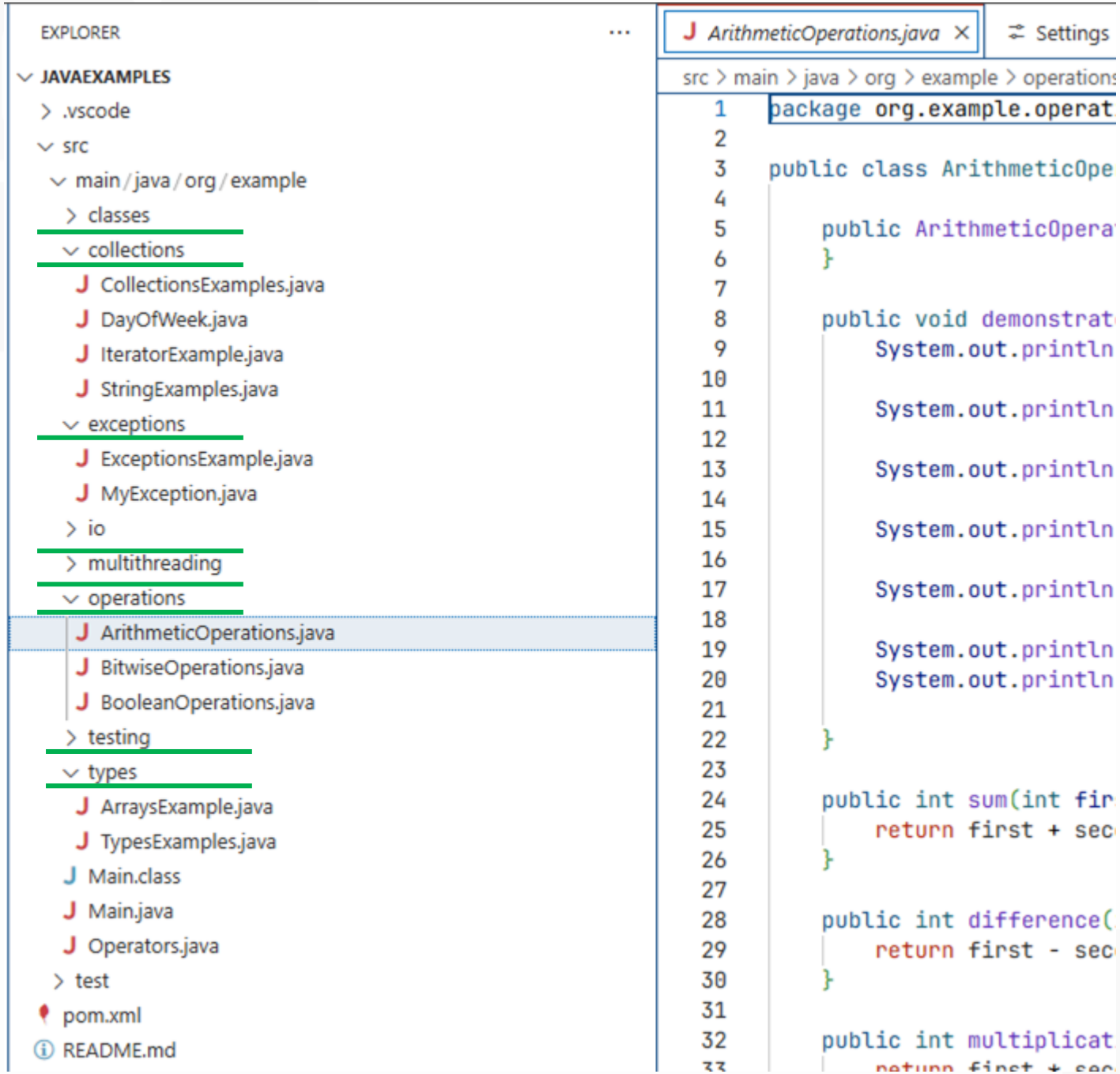
null показывает, что значение пока отсутствует.

4. Возврат из методов

Метод может вернуть null, если объект не найден:

```
User findUser() {
    return null;
}
```

Например, поиск пользователя по имени может не найти результат.



ПАКЕТЫ

- ***Пакет (package)*** – пространство имен в Java
- Пакет объединяет ***типы*** (классы, интерфейсы, перечисления), относящиеся к одной предметной области или одной задаче

IThub

Организация классов в виде пакетов позволяет избежать конфликта имен между классами. Ведь нередки ситуации, когда разработчики называют свои классы одинаковыми именами. Принадлежность к пакету позволяет гарантировать однозначность имен.

Как правило, названия пакетов соответствуют физической структуре проекта, то есть организации каталогов, в которых находятся файлы с исходным кодом. А путь к файлам внутри проекта соответствует названию пакета этих файлов.

IThub

Модификаторы доступа

IThub

- **public (открытый)**

Доступ к нему может получить любой другой код (видны другим классам из текущего пакета и из внешних пакетов)

- **private (закрытый)**

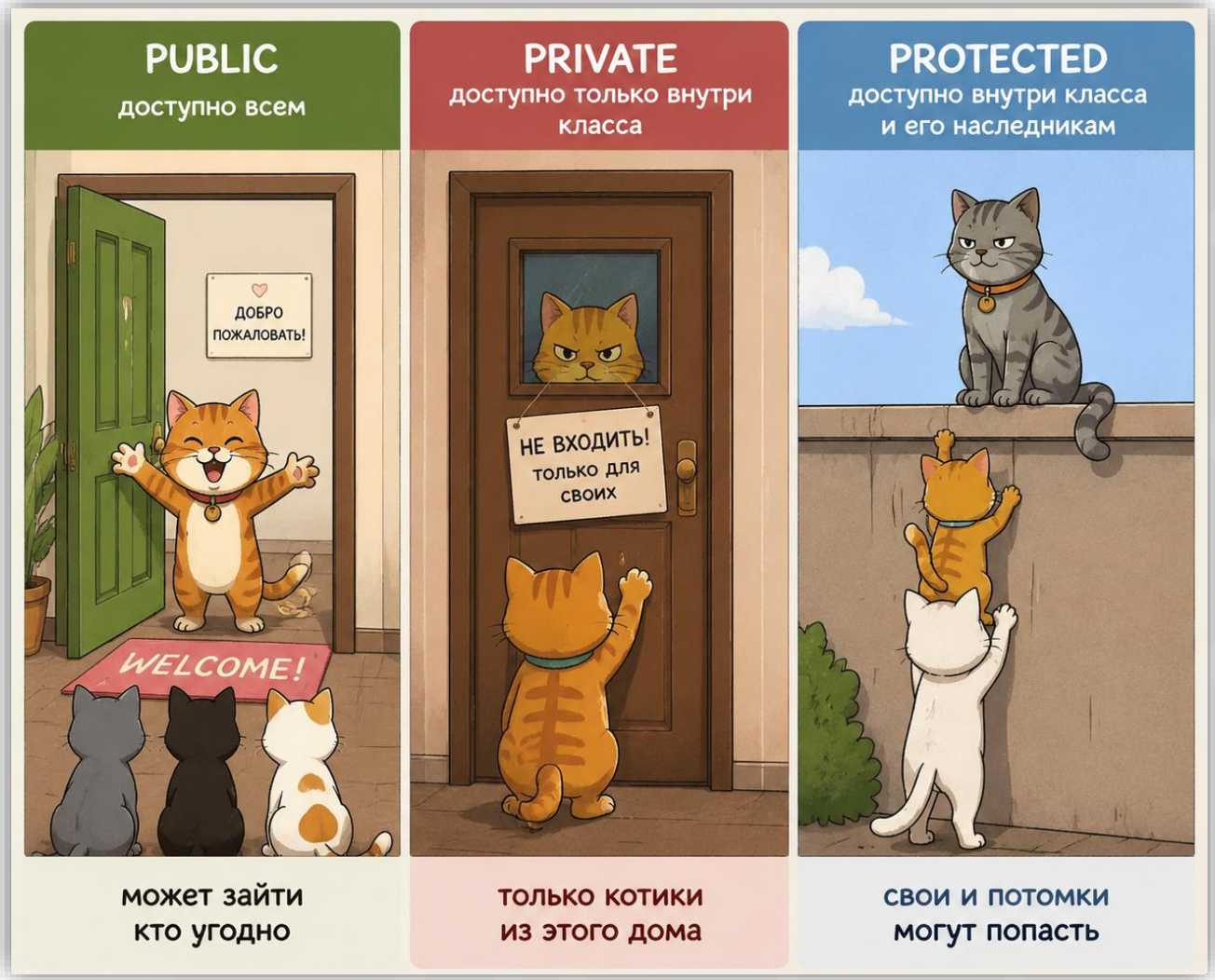
Доступ к нему могут получать только другие члены этого класса (только из кода в том же классе)

- **protected (защищенный)**

Такой класс или член класса доступен из любого места в текущем классе или пакете или в производных классах, даже если они находятся в других пакетах

- **модификатор по умолчанию**

Отсутствие модификатора у поля или метода класса предполагает применение к нему модификатора по умолчанию. Такие поля или методы видны всем классам в текущем пакете.



```java
public class Box {

    private double width;
    private double height;
    private double depth;

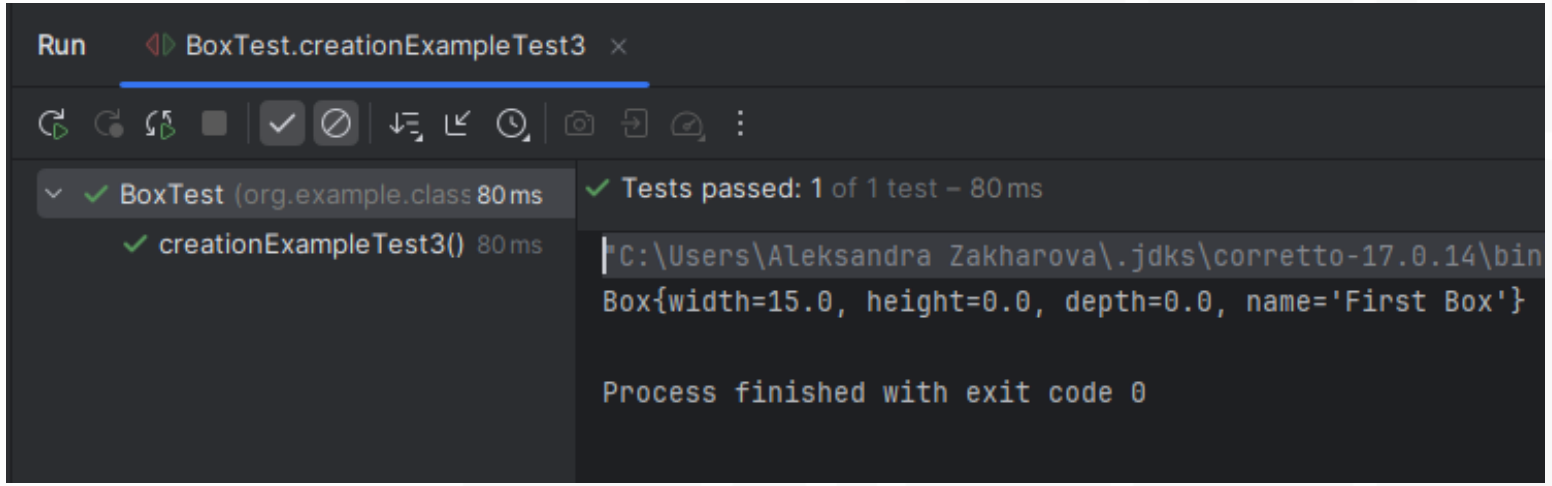
    protected String name;

    ...

    @Override
    public String toString() {
        return "Box{" +
                "width=" + width +
                ", height=" + height +
                ", depth=" + depth +
                ", name='" + name + '\'' +
                '}';
    }
```

```java
    @Test
    public void creationExampleTest3() {
        Box box = new Box();
        //box.width = 15; // нельзя инициализировать private поле
        box.setWidth(15);

        box.name = "First Box";
        System.out.println(box);
    }
```



IThub

Ключевое слово new

Ключевое слово new используется для создания нового объекта

Что делает new?

1. Выделяет память - создаёт место в памяти для будущего объекта.
2. Вызывает конструктор - запускает метод, который инициализирует объект.
3. Возвращает ссылку - передаёт адрес созданного объекта, чтобы с ним можно было работать.

IThub

Ключевое слово final

Предотвращает изменение содержимого, делая по существу константой

```
package java.lang;

…

public final class Math {


    private Math() {}

    public static final double E = 2.7182818284590452354;

    public static final double PI = 3.14159265358979323846;
```

IThub

Задание

IThub

1. Создайте класс Player
Добавьте private поля и создайте конструктор:
- имя игрока
- уровень
- количество здоровья

2. Создайте геттеры и сеттеры:
getName()
setName()

getLevel()
setLevel()

getHealth()
setHealth ()

3. Добавьте методы класса:
- Метод вывода информации **showInfo**() должен выводить: Игрок: Алекс Уровень: 5 Здоровье: 100
- Метод получения урона **takeDamage**() принимает количество урона и уменьшает здоровье игрока. (Например: игрок получил 30 урона осталось здоровья: 70)
- Метод повышения уровня **levelUp**() увеличивает уровень игрока на 1

4. В классе Main создайте объект игрока
Задайте ему значения: имя: "Алекс" уровень:5 здоровье: 100
Выполните действия:
выведите информацию об игроке
Нанесите ему 30 урона
Повышайте уровень
Снова выведите информацию